

Latency-Optimized Architecture

For a Clustered-Server English Communication Coaching Chatbot

This guide revises the original single-pair two-model (Qwen3 8B + DeepSeek-R1 8B) local architecture for a new context: the models run on clustered servers rather than a single 8 GB consumer GPU, the product is focused on student English and communication coaching, and the system must support recorded speech exercises in addition to text chat. The goal is to reduce average response latency without giving up accuracy on the turns where it matters.

Scope note: serving-layer infrastructure optimizations (batched inference engines such as vLLM, prefix/KV caching, speculative decoding) are intentionally left out of this document, since they are an infrastructure concern rather than an application architecture concern. The one exception is prompt-prefix caching, which is called out below because it is cheap to add from day one and directly reduces latency for every request without any architectural change.

1. Why the Original Design Adds Latency

The original pipeline runs every turn through up to three sequential LLM calls: Qwen drafts an answer, DeepSeek-R1 verifies it, and Qwen corrects it if the verifier disagrees. Two issues compound on a coaching product:

- DeepSeek-R1 is a reasoning model. It generates long chain-of-thought before answering, which is slow and mostly wasted on checking whether a grammar correction or a casual reply makes sense.
- Most turns in an English-practice conversation are low-stakes (small talk, simple corrections, encouragement) and do not need a full fact-verification pass at all.

The revised design below keeps the draft → verify → correct pattern only where it earns its cost, and gives each exercise type (conversation, grammar analysis, speech evaluation, factual Q&A) its own lighter-weight path.

2. Revised Core Architecture

A router classifies each text turn by complexity/topic before anything else runs. Casual conversation and simple corrections go straight to a small, fast model. Only complex or factual queries enter the full draft/verify/correct chain with a larger model pair.

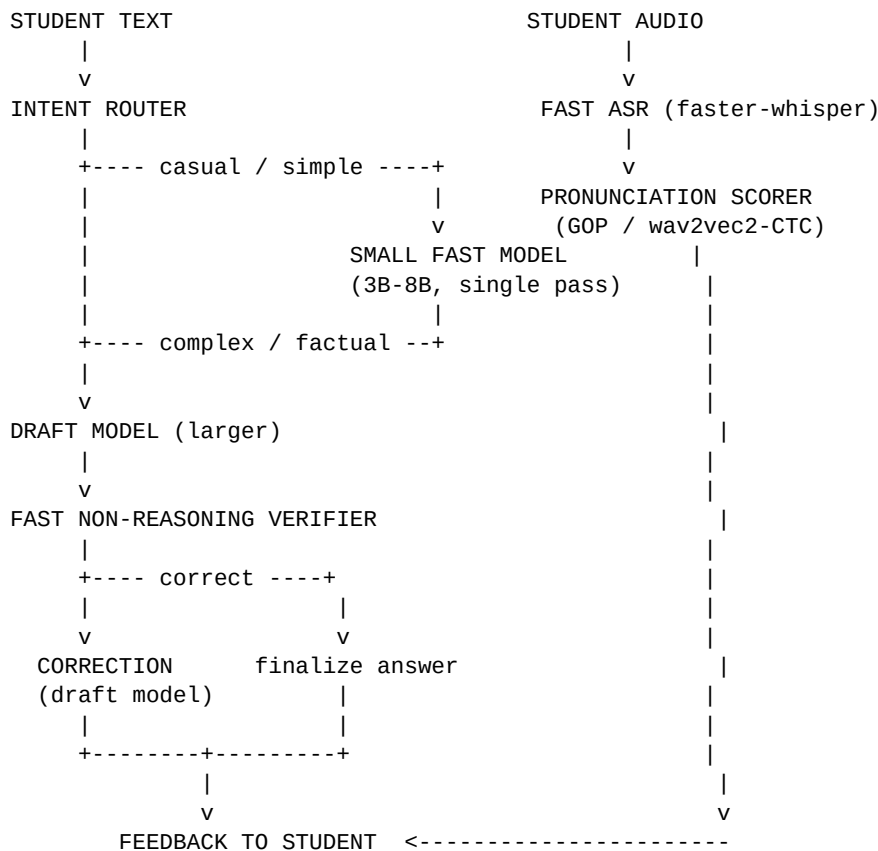


Figure 1. Routed architecture: casual turns take a single fast pass; complex or factual turns keep the draft/verify/correct chain; speech is scored separately before joining the text flow.

2.1 Conditional verification

Only invoke the verifier when the router flags a turn as factual, current-information, or otherwise high-stakes. Casual conversation, encouragement, and simple grammar replies skip verification entirely. Since most turns in a coaching conversation are low-stakes, this alone removes one or two LLM calls from the majority of turns.

2.2 Replace the reasoning-model verifier

Swap DeepSeek-R1 out of the verifier role. A non-reasoning instruct model (or a smaller fine-tuned classifier for straightforward checks) is far faster for the narrow job of checking relevance, accuracy, and tone. Reserve any reasoning-heavy model for genuinely hard verification cases only, if at all.

2.3 Cascade model sizes instead of a fixed pair

Route by difficulty rather than always calling two similarly-sized models:

- Small/fast model (roughly 3B–8B class): casual conversation, simple grammar replies, pronunciation feedback text.
- Larger model: complex reasoning, essay-level feedback, current-information queries that need the full draft/verify/correct chain.

3. Speech Evaluation Pipeline

Speech evaluation should not go through the chat LLMs for its acoustic component. Decouple it into a dedicated pipeline that only hands the LLM a compact, structured result:

- **ASR:** faster-whisper (a CTranslate2-optimized Whisper) or distil-whisper for transcription speed close to real time, with accuracy comparable to full Whisper.
- **Pronunciation / fluency scoring:** a dedicated model, e.g. Goodness-of-Pronunciation (GOP) scoring via forced alignment (Montreal Forced Aligner / Kaldi), or a wav2vec2-CTC-based phoneme scorer.
- **LLM step:** only the transcript and the numeric/categorical scores are passed to the small fast model, which turns them into human-readable feedback. The LLM never processes raw audio directly.

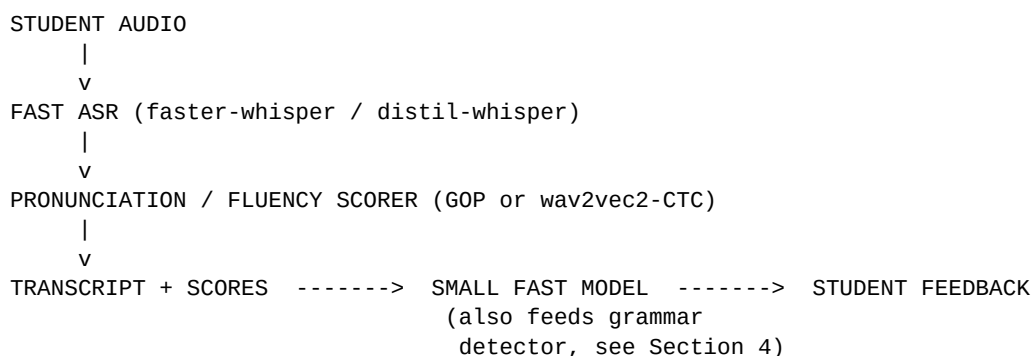


Figure 2. Speech pipeline stays acoustic-only until the final feedback step, keeping the expensive LLM call short: text in, feedback out.

4. Grammar Analysis Pipeline

Grammar-analysis exercises do not need an LLM to find the errors. A dedicated grammar error correction (GEC) tool is faster and less error-prone than asking an LLM to spot every mistake from scratch.

4.1 Detector

Use GECToR (a sequence-tagging model that predicts edit operations in a single forward pass, rather than generating corrected text token-by-token) or LanguageTool (rule- and statistics-based, self-hostable). Output: a structured list of error spans, each with an error category (subject–verb agreement, tense, article, preposition, etc.) and a suggested fix. This is deterministic, fast, and does not hallucinate errors that are not there.

4.2 Explainer

The small fast model receives only the structured error list and writes a friendly, pedagogical explanation of each flagged error. Because the model is explaining pre-identified errors rather than hunting for them, this call is short, low-latency, and much less prone to missing or inventing issues.

4.3 No verify loop needed

Because the deterministic detector already functions as ground truth, grammar exercises can skip the draft/verify/correct chain entirely — the detector output *is* the verification.

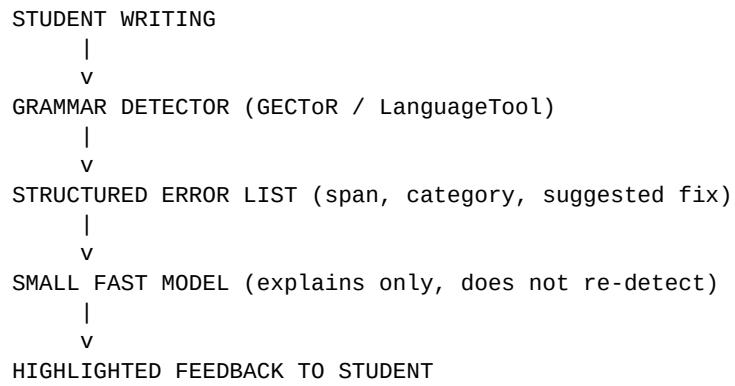


Figure 3. Detection and explanation are split: a deterministic tool finds errors, a small model only explains them.

This structured error output can also be logged over time as a per-student error-tracking signal (e.g. "article errors down 50% this month") at no extra LLM cost.

5. Unified Routing Summary

Exercise type	Primary path	Verification	Typical latency driver
Casual conversation	Small fast model, single pass	None	One short generation
Complex / factual Q&A	Draft model → fast verifier → correction if needed	Fast non-reasoning verifier	Up to 2–3 calls, only when flagged
Grammar analysis	Deterministic GEC detector → small model explainer	Detector output is the verification	One deterministic pass + one short generation
Speech evaluation	ASR → pronunciation scorer → small model feedback	Scoring model output is the verification	Acoustic pipeline runs before any LLM call

6. What's Intentionally Left Out For Now

Serving-layer optimizations — batched inference engines (e.g. vLLM, TensorRT-LLM), continuous batching, and speculative decoding — are left out of this revision, since they are infrastructure decisions that can be layered on once the application architecture above is working, without requiring changes to the routing logic or exercise pipelines.

Exception worth doing early: prompt-prefix / KV caching for your system prompts. Your prompts (Qwen draft prompt, verifier prompt, correction prompt) are long and repeated on every call. Most serving stacks cache the prefix automatically once you standardize prompt structure, so it is worth keeping your system prompts stable and reused verbatim from day one — that habit costs nothing now and pays off immediately once a caching-aware server is introduced later.

7. Suggested Build Order

- Stand up the intent router with a simple rule/classifier split: casual vs. complex/factual.
- Wire the small fast model for casual conversation (no verification).
- Wire the larger draft model + fast non-reasoning verifier for complex/factual turns.
- Add the grammar detector (GECToR or LanguageTool) and the explanation-only LLM call.
- Add the ASR + pronunciation scoring pipeline, feeding transcripts into the grammar detector and text pipeline.
- Instrument latency per path so you can see which lane is actually slow before reaching for serving-layer optimizations.

Note: this document is an architecture guide, not a deployment spec. Model tags, library APIs, and available tooling can change — verify current versions before implementation.